JTAGprobe_smp_pio_std_weact — WeAct RP2040 16 MB: Configuration, Build and Differences

CMSIS-DAP v2 JTAG/SWD probe with a USB CDC-ACM UART bridge, built for the **WeAct Studio RP2040 Core Board V1.0 (16 MB QSPI flash)**, as a drop-in replacement for the `JTAGprobe_smp_pio_std` firmware that runs on a standard Raspberry Pi Pico.

Document status. Written on 2026-10-25, after the WeAct build was fixed and verified on hardware. JTAG (OpenOCD) and the UART bridge work on the same target and the same cable that the Pico build uses.

Repository state. The tree is at commit `e17b7b0` plus these local changes, all of which this document describes:

- `CMakeLists.txt` — board selection, target name, and the removal of the earlier `PICO_STDIO_UART OFF` override (section 8).
- `include/DAP_config.h` — explicit nTRST/nRESET initialisation in `DAP_SETUP()` (section 4.4).

Toolchain used for the shipped build. xPack GNU Arm Embedded GCC 15.2.1 (`20251203`) against Raspberry Pi Pico SDK 2.3.0. The known-good Pico binary in `../JTAGprobe_smp_pio_std/build/` was built with Arch GCC 16.2.0; see section 7.6 for why that difference matters when comparing the two binaries.

> **Important.** `weact.md` / `weact.pdf` in this directory are the notes from the first investigation. Their section 2.1 ("disable stdio UART") describes the change that actually broke JTAG. Section 8 of this document supersedes it.

---

1. Scope

This document covers:

1. the WeAct board as it is used by the probe (pinout, on-board loads, differences from the Raspberry Pi Pico);
2. the firmware configuration that is specific to this board;
3. how to reproduce the build, verify it, flash it, and use it;
4. a precise comparison with the `JTAGprobe_smp_pio_std` tree, including the compiled binaries;
5. the root cause of the original "JTAG scan chain interrogation failed: all ones" failure and the fix that resolved it.

It does not repeat the internal design of the firmware. For that, see `ARCHITECTURE_AND_IMPLEMENTATION.md` in this directory.

---

2. Hardware

2.1 Board summary

Feature	WeAct Studio RP2040 Core Board V1.0 (16 MB)
MCU	RP2040, dual Cortex-M0+, 133 MHz, 264 KB SRAM
Flash	16 MB QSPI (W25Q128-class; the schematic footprint is W25Q32JVSS)
Crystal	12 MHz
USB	USB-C, 5.1 kΩ CC pull-downs on CC1/CC2
3.3 V supply	SC662K-3.3 LDO, 250 mA max, input tied to VSYS

Feature	WeAct Studio RP2040 Core Board V1.0 (16 MB)
User LED	`LED1` on **GPIO25**, active high, 1 kΩ to GND
Power LED	`LED2`, on +3.3 V
User key	`KEY` on **GPIO23**, 1 kΩ series to GND, 1 kΩ pull-up to 3.3 V
Buttons	`BOOTSEL` on QSPI_SS, `NRST` on RUN
Debug header `J2`	4 pins: 3V3, SWDIO, SWCLK, GND (RP2040's own debug port)
Headers	2 × 20, 2.54 mm pitch
Size	21.52 mm × 53.00 mm
`3V3_EN`	Not connected (pin 37 of the Pico footprint is NC)

The firmware does not drive GPIO23, GPIO24, GPIO26–GPIO29, or the debug port. Everything the probe uses is on the left header, pins 1–10.

2.2 Left header `J1` (the probe connector) — identical to the Pico

Physical order, USB-C at the top, component side up. The WeAct schematic numbers `J1` from the bottom (pin 1 = GP15, pin 20 = GP0), but the physical positions are the same as on a Pico:

Position	Pico pin	WeAct `J1` pin	Signal
1	1	20	**GPIO0**
2	2	19	**GPIO1**
3	3	18	GND
4	4	17	**GPIO2**
5	5	16	**GPIO3**
6	6	15	**GPIO4**
7	7	14	**GPIO5**
8	8	13	GND
9	9	12	**GPIO6**
10	10	11	**GPIO7**
11	11	10	GPIO8
12	12	9	GPIO9
13	13	8	GND
14	14	7	GPIO10
15	15	6	GPIO11
16	16	5	GPIO12
17	17	4	GPIO13

Position	Pico pin	WeAct `J1` pin	Signal
18	18	3	GND
19	19	2	GPIO14
20	20	1	GPIO15

Consequences: a 10-way ribbon that works on a Pico's left header works on the WeAct without any adapter, in the same orientation. The two GND pins on the probe use positions 3 and 8, and both boards have GND there.

2.3 Right header `J3` — differs from the Pico

The right header is **not** position-compatible with the Pico. Only GND (positions 3, 8, 13, 18) and ADC_VREF (position 6) are at the same places:

Position	Raspberry Pi Pico	WeAct RP2040 16 MB
1	VBUS	VBUS
2	VSYS	VSYS
3	GND	GND
4	3V3_EN	**3V3**
5	3V3	**GPIO24**
6	ADC_VREF	ADC_VREF
7	GPIO28	**GPIO29**
8	GND	GND
9	GPIO27	**GPIO28**
10	GPIO26	GPIO27
11	RUN	**GPIO26**
12	GPIO22	GPIO22
13	GND	GND
14	GPIO21	GPIO21
15	GPIO20	GPIO20
16	GPIO19	GPIO19
17	GPIO18	GPIO18
18	GND	GND
19	GPIO17	GPIO17
20	GPIO16	GPIO16

> **Warning.** A Pico-shaped adapter that takes 3.3 V from Pico pin 36 (position 5 of the right header) gets **GPIO24** on the WeAct, not 3.3 V. Take 3.3 V from the WeAct's position 4 instead. GND and ADC_VREF are safe. The probe itself uses only the left header pins 1–10.

2.4 Probe signal map

The firmware pin numbers are the same as in `JTAGprobe_smp_pio_std` (`include/DAP_config.h`, `include/DAP.h`, `include/board_pico_config.h`):

Header position	GPIO	Signal	Direction	Implementation
1	0	**nTRST**	Probe → target	SIO output, driven high at boot and when JTAG starts; not host-controllable
2	1	**nRESET**	Open-drain	SIO input with pull-up (released); drive low to assert
3	—	GND	—	—
4	2	**SWCLK / TCK**	Probe → target	PIO side-set 0; SIO for bit-banged commands
5	3	**SWDIO / TMS**	Bi-directional (SWD) / output (JTAG)	PIO SET 0 in JTAG; SIO in SWD
6	4	**UART TX**	Probe → target	UART1 TX, 115200 8N1 default
7	5	**UART RX**	Probe ← target	UART1 RX
8	—	GND	—	—
9	6	**TDI**	Probe → target	PIO OUT 0
10	7	**TDO**	Probe ← target	PIO IN 0; SIO input with pull-up, input synchroniser bypassed
on-board	25	**Status LED**	Output	SIO, 1 kHz timer IRQ on core 0

JTAG is clocked by a PIO state machine (`src/jtag.pio`), four PIO cycles per TCK period, with `div = f_sys / (4 × f_TCK)`; `DAP_SWJ_Clock` (OpenOCD `adapter speed`) updates the divider. SWD and the non-sequence JTAG commands bit-bang through SIO.

2.5 On-board connections relevant to the probe

Verified on `HDK/sch/WeAct-RP2040-V10 SchDoc.pdf` and `HDK/PINout.PNG`:

- `GPIO0`…`GPIO7` run directly from the RP2040 to `J1`, with no series resistors, no ESD parts and no other on-board loads.
- The 16 MB QSPI flash is on the RP2040's **dedicated** QSPI pins (`QSPI_SS`, `QSPI_SCLK`, `QSPI_SD0`…`SD3`); it does not share `GPIO0`–`GPIO5`.

- `GPIO25` goes only to `LED1` through 1 kΩ. It is a plain LED, not a WS2812, so the 10 Hz blink is visible.
- `GPIO23` goes only to the `KEY` button (1 kΩ to GND, 1 kΩ pull-up).
- `J2` exposes the RP2040's own SWD debug port (dedicated `SWCLK` / `SWDIO` pins), not `GPIO2` / `GPIO3`.

> **Warning.** `J2` is a 4-pin header in the order 3V3, SWDIO, SWCLK, GND. The Pico's debug header is 3 pins in the order SWCLK, GND, SWDIO. Do not plug a Pico SWD cable into `J2` : its GND would land on SWDIO and its SWCLK would land on 3.3 V. This is unrelated to the probe's JTAG output, which is on `J1` pins 1–10.

2.6 Mechanical and electrical notes

- The board is 53.0 mm long versus the Pico's 51 mm and 21.52 mm wide versus 21 mm. Headers are on a standard 2.54 mm grid, so a Pico socket/adapter still seats; the extra 2 mm only overhangs.
- The 3.3 V regulator is a 250 mA LDO (not the Pico's SMPS). If the target is powered from the probe's 3.3 V, keep the target's current draw well below that; the probe itself consumes only a few tens of mA.
- There is no `3V3_EN` pin and no `RUN` pin on the right header. Reset the probe with the on-board `NRST` button or by unplugging USB.

3. Firmware overview

The firmware is unchanged from `JTAGprobe_smp_pio_std`; this section is only a map for the rest of the document.

Layer	What it does
µOS++ III SMP	Kernel; `main` thread on core 0, `usb` and `uart` threads on core 0, `dap` thread alone on core 1
USB	TinyUSB, `OPT_OS_NONE`, all calls serialised by the probe's own mutex; CMSIS-DAP v2 bulk (512-byte packets, 4 buffers) + CDC-ACM
DAP	CMSIS-DAP command layer; `DAP_JTAG_Sequence` is streamed to the PIO, other commands bit-bang or use SWD
PIO JTAG	`src/jtag.pio` + `src/pio_jtag.c`; TMS is carried in the stream, so the CPU never waits between entries
UART bridge	CDC line coding applied to UART1 on GP4/GP5; 32-byte FIFO batches
Status LED	GP25: steady on when idle; 10 Hz blink while TCK/SWCLK is active (50 ms toggle, 100 ms hold after the last clocking command)

USB identity (unchanged): VID:PID `2e8a:000c`, product string `JTAG probe on Pico (CMSIS-DAP)`, serial number derived from the flash unique ID. The "Pico" in the product string is historical; it is cosmetic and does not affect operation.

4. Configuration

4.1 `CMakeLists.txt`

The board-specific configuration is at the top of the file:

```
cmake_minimum_required(VERSION 3.20)

# Default to the 16 MB WeAct Studio RP2040 Core Board (LED on GP25, 16 MB QSPI).
set(PICO_BOARD weact_studio_rp2040_16mb CACHE STRING "Board type")

# Keep the SDK's stdio UART: on the working Pico build board_init() places
# UART0 TX on GP0 (nTRST) and RX on GP1 (nRESET), and UART0 TX idles high.
# That is what holds nTRST released from boot until JTAG claims the pins.
# DAP_SETUP() now also drives nTRST high explicitly (DAP_config.h).

include(pico_sdk_import.cmake)

project(JTAGprobe_smp_pio_std_weact C CXX ASM)
```

The rest of the file is the same as in the Pico tree, with the target renamed to `JTAGprobe_smp_pio_std_weact`. The important properties are:

- `TINYUSB_OPT_OS=OPT_OS_NONE` is set before `pico_sdk_init()`, so TinyUSB does not use the SDK's own lock;
- `PICO_RP2040_USB_DEVICE_ENUMERATION_FIX=1` is defined;
- `OS_USE_OS_APP_CONFIG_H` selects `src/cmsis-plus/os-app-config.h`;
- `PICO_USE_MALLOC_MUTEX=0` because the µOS++ allocator already runs under the scheduler lock on both cores;
- the binary type is `copy_to_ram`;
- libraries: `micro-os-plus::rp2040`, `pico_stdlib`, `pico_unique_id`, `tinyusb_device`, `tinyusb_board`, `hardware_pio`.

> **Do not set `PICO_STDIO_UART OFF` on this design.** That was the change that broke JTAG; see section 8. Keep the SDK default (ON). It is what configures UART0 on GP0/GP1 and therefore holds nTRST high from boot. The explicit nTRST drive added in `DAP_SETUP()` makes the design robust even if somebody changes this later, but the two together are the tested state.

4.2 Board header: `weact_studio_rp2040_16mb.h`

`PICO_BOARD=weact_studio_rp2040_16mb` selects this header from the SDK (`$PICO_SDK_PATH/src/boards/include/boards/`). The settings that matter:

```
// --- UART ---
#define PICO_DEFAULT_UART         0
#define PICO_DEFAULT_UART_TX_PIN  0
#define PICO_DEFAULT_UART_RX_PIN  1

// --- LED ---
#define PICO_DEFAULT_LED_PIN      25

// --- I2C ---
#define PICO_DEFAULT_I2C          0
#define PICO_DEFAULT_I2C_SDA_PIN  4
#define PICO_DEFAULT_I2C_SCL_PIN  5

// --- SPI ---
#define PICO_DEFAULT_SPI          0
#define PICO_DEFAULT_SPI_SCK_PIN  18
#define PICO_DEFAULT_SPI_TX_PIN   19
#define PICO_DEFAULT_SPI_RX_PIN   16
#define PICO_DEFAULT_SPI_CSN_PIN  17

// --- FLASH ---
#define PICO_BOOT_STAGE2_CHOOSE_W25Q080 1
#define PICO_FLASH_SPI_CLKDIV           2
#define PICO_FLASH_SIZE_BYTES           (16 * 1024 * 1024)
```

Compared with the Pico board header (`pico.h`), the **UART0 pins, the LED pin (25), the I2C and SPI defaults, the boot2 choice and the QSPI divider are identical**. The meaningful difference is the flash size: 16 MB instead of 2 MB. With `copy_to_ram` the flash is only read by the second-stage boot loader to copy the image to RAM; the 16 MB part is addressed with the normal 24-bit QSPI addresses, so no special boot2 or 4-byte addressing is needed.

The I2C defaults on GP4/GP5 do not conflict with the firmware: nothing initialises I2C, and the probe uses GP4/GP5 as UART1.

4.3 Build-time definitions

These come from the board header, the SDK and `CMakeLists.txt`:

Definition	Value / meaning
`PICO_BOARD`	`"weact_studio_rp2040_16mb"`
`PICO_RP2040`	1
`PICO_FLASH_SIZE_BYTES`	16,777,216
`PICO_BOOT_STAGE2_CHOOSE_W25Q080`	1 (same boot2 as the Pico)
`PICO_FLASH_SPI_CLKDIV`	2 (62.5 MHz QSPI at 125 MHz sysclk)
`PICO_COPY_TO_RAM`	1
`PICO_NO_FLASH`	0
`CFG_TUSB_MCU`	`OPT_MCU_RP2040`
`CFG_TUSB_OS`	`OPT_OS_NONE`
`PICO_RP2040_USB_DEVICE_ENUMERATION_FIX`	1
`PICO_RP2040_USB_DEVICE_UFRAME_FIX`	1

Definition	Value / meaning
`OS_USE_OS_APP_CONFIG_H`	1
`OS_USE_SMP_SCHEDULER`, `OS_NCPU`	1, 2
`PICO_USE_MALLOC_MUTEX`	0
`LIB_PICO_STDIO_UART`	1 (must stay defined)
`PICO_TARGET_NAME`	`"JTAGprobe_smp_pio_std_weact"`

Of the board-header settings, `PICO_FLASH_SIZE_BYTES` is the only one that changes the linked image: it sets the `FLASH` region length in the linker script and the value recorded in the binary information block. The other settings (UART0 pins, LED pin, I2C/SPI defaults) match the Pico and do not change for this firmware.

4.4 DAP pin setup and the nTRST/nRESET initialisation

The pin numbers are unchanged from the working Pico tree:

```
// include/DAP_config.h
#define PROBE_PIN_SWCLK_TCK   2
#define PROBE_PIN_SWDIO_TMS   3
#define PROBE_PIN_TDI         6
#define PROBE_PIN_TDO         7
#define PROBE_PIN_nTRST       0
#define PROBE_PIN_nRESET      1
```

`DAP_SETUP()` in this tree additionally leaves nTRST released and nRESET de-asserted from the very first call:

```
   gpio_set_dir(PROBE_PIN_TDO,false);

   gpio_set_dir(PROBE_PIN_nRESET,false);
   gpio_put(PROBE_PIN_nRESET,true);
+  gpio_pull_up(PROBE_PIN_nRESET);
+
+  /* Release nTRST (high) from boot.  On the Pico build UART0 TX on GP0 does
+     this implicitly after DAP_SETUP(); doing it here keeps the target TAP out
+     of reset on boards whose stdio configuration leaves GP0 alone. */
+  gpio_put(PROBE_PIN_nTRST,true);
+  gpio_set_dir(PROBE_PIN_nTRST,true);
```

The order matters: `gpio_init()` has already set the output latch to 0, so the latch is written to 1 **before** the output enable is turned on. GP0 goes from high-Z to high without a low glitch. `pio_jtag_init()` independently does the same when JTAG starts (`src/pio_jtag.c`), so the pin state is the same before and during a session.

nRESET remains open-drain: latch 0 + output = asserted (low), input + pull-up = released. `PORT_OFF()` puts both pins back to high-Z with the pulls disabled; because `PIN_nTRST_OUT()` is a stub, only the next `PORT_JTAG_SETUP()` (and now `DAP_SETUP()`) re-establishes the high level. This is the same behaviour as the working Pico build.

4.5 CMake cache gotcha

`PICO_STDIO_UART` is a cache variable. If a build directory was previously configured with the old `set(PICO_STDIO_UART OFF ... FORCE)` line, the value `OFF` stays in `CMakeCache.txt` even after the line is removed. Either reconfigure explicitly:

```
cmake -S . -B build -DPICO_STDIO_UART=ON
```

or start from a fresh build directory. Check with:

```
grep PICO_STDIO_UART build/CMakeCache.txt     # expect :BOOL=ON
grep -c "LIB_PICO_STDIO_UART=1" build/build.ninja   # expect a non-zero count
```

5. Building

5.1 Prerequisites

- Raspberry Pi Pico SDK **2.3.0** (the SDK in this workspace is `/home/dan/PICO/pico-sdk`, commit `98a542c1`, "SDK 2.3.0 Release").
- CMake 3.20 or newer.
- Ninja (recommended) or Make.
- An ARM bare-metal GCC toolchain. This build used **xPack GNU Arm Embedded GCC 15.2.1** from `/home/dan/.local/xPacks/@xpack-dev-tools/arm-none-eabi-gcc/15.2.1-1.1.1/.content`.
- `picotool` is downloaded by the SDK's build system if it is not found (`build/_deps/picotool`), so no manual install is required for a UF2 build.
- The µOS++ kernel and its RP2040 port are already in `micro-os-plus/`; no submodules are fetched at build time.

5.2 Toolchain selection

The build uses whichever `arm-none-eabi-gcc` is first in `PATH`. To pin it explicitly (recommended for reproducible builds):

```
T=/home/dan/.local/xPacks/@xpack-dev-tools/arm-none-eabi-gcc/15.2.1-1.1.1/.content
export PICO_TOOLCHAIN_PATH="$T"
```

or pass the compiler paths to CMake (section 5.6). The delivered binary was built with this compiler; its `.comment` section reports:

```
GCC: (xPack GNU Arm Embedded GCC x86_64) 15.2.1 20251203
```

For comparison, the known-good Pico binary reports Arch GCC 16.2.0. Both are newer than the versions the SDK is normally validated with; keeping **one** toolchain for both trees makes A/B comparisons meaningful.

5.3 Configure and build

From `/home/dan/Downloads/pico-jtag/JTAGprobe_smp_pio_std_weact`:

```
export PICO_SDK_PATH=/home/dan/PICO/pico-sdk
export PATH=/home/dan/.local/xPacks/@xpack-dev-tools/arm-none-eabi-gcc/15.2.1-1.1.1/.content/bin:
        $PATH

cmake -S . -B build -G Ninja \
      -DCMAKE_BUILD_TYPE=Release \
      -DPICO_BOARD=weact_studio_rp2040_16mb \
      -DPICO_STDIO_UART=ON

cmake --build build -j"$(nproc)"
```

`PICO_BOARD` is already set in `CMakeLists.txt`, so `-DPICO_BOARD=...` is redundant but explicit. `-DPICO_STDIO_UART=ON` is only needed when reusing a build directory that was configured with the old override; a fresh build directory defaults to ON.

5.4 Outputs

In `build/`:

File	Size (bytes)	Contents
`JTAGprobe_smp_pio_std_weact.uf2`	120,320	Flash image for the RP2040 bootloader
`JTAGprobe_smp_pio_std_weact.bin`	60,108	Raw flash image (boot2 + `copy_to_ram` image)
`JTAGprobe_smp_pio_std_weact.elf`	1,803,708	ELF with debug information
`JTAGprobe_smp_pio_std_weact.hex`	169,121	Intel HEX
`JTAGprobe_smp_pio_std_weact.dis`	1,158,877	Disassembly
`JTAGprobe_smp_pio_std_weact.elf.map`	781,558	Linker map

`arm-none-eabi-size` reports `text = 64,204`, `data = 0`, `bss = 40,260`: about 64 KB of code and 40 KB of BSS. The image runs from RAM (`copy_to_ram`), so the 16 MB flash is irrelevant to the runtime footprint; only boot2 reads it.

5.5 Verify the build

```
# Compiler actually used
readelf -p .comment build/JTAGprobe_smp_pio_std_weact.elf

# The SDK stdio UART must be linked (it drives GP0/GP1)
grep -c "LIB_PICO_STDIO_UART=1" build/build.ninja

# Size summary
arm-none-eabi-size build/JTAGprobe_smp_pio_std_weact.elf
```

Optional: confirm that `DAP_SETUP()` drives nTRST high and pulls up nRESET. The exact addresses vary with the build, so locate the symbol first:

```
NM=arm-none-eabi-nm; OBJ=arm-none-eabi-objdump
$NM -S build/JTAGprobe_smp_pio_std_weact.elf | grep " DAP_Setup$"
# Disassemble the size printed by nm -S; look for the SIO writes:
#   str r5,[r4,#20]   -> SIO GPIO_OUT_SET = bit 0 (GP0 latch = 1)
#   str r5,[r4,#36]   -> SIO GPIO_OE_SET  = bit 0 (GP0 output)
#   bl gpio_set_pulls -> GP1 pull-up (r0=1, r1=1, r2=0)
```

5.6 Clean rebuild and switching compilers

```
rm -rf build
# then re-run the configure and build commands from 5.3
```

To pin a specific compiler (example: the 14.2.1 toolchain also available on this machine):

```
T=/home/dan/.local/xPacks/@xpack-dev-tools/arm-none-eabi-gcc/14.2.1-1.1.1/.content
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release \
      -DPICO_SDK_PATH=/home/dan/PICO/pico-sdk \
      -DPICO_TOOLCHAIN_PATH="$T" \
      -DCMAKE_C_COMPILER="$T/bin/arm-none-eabi-gcc" \
      -DCMAKE_CXX_COMPILER="$T/bin/arm-none-eabi-g++" \
      -DCMAKE_ASM_COMPILER="$T/bin/arm-none-eabi-gcc"
cmake --build build -j"$(nproc)"
```

CMake refuses to switch the compiler inside an existing build directory; delete it first.

---

6. Flashing and first use

6.1 Flash the UF2

1. Hold `BOOTSEL` and connect the board with the USB-C cable. A mass-storage volume named `RPI-RP2` appears.
2. Copy `build/JTAGprobe_smp_pio_std_weact.uf2` to it. The board reboots automatically.
3. If `picotool` is installed, the equivalent is:

   ```
   picotool load -f build/JTAGprobe_smp_pio_std_weact.uf2
   ```

The 16 MB flash size does not change the flashing procedure; the UF2 family and load addresses are the standard RP2040 ones.

6.2 USB enumeration

The probe enumerates as a composite device:

- **Interface 0** — vendor-specific bulk: CMSIS-DAP v2, used by OpenOCD, pyOCD and probe-rs through libusb. It does **not** appear as `/dev/tty*`.
- **Interfaces 1–2** — CDC-ACM: a `/dev/ttyACMx` on Linux (`COMx` on Windows), the UART bridge to GP4/GP5.

Check with:

```
lsusb | grep 2e8a:000c
dmesg | tail
ls -l /dev/serial/by-id/
```

The serial number is derived from the RP2040's flash unique ID, so two probes with the same VID:PID can be distinguished by serial.

6.3 OpenOCD

```
openocd -f interface/cmsis-dap.cfg \
        -c "transport select jtag" \
        -c "adapter speed 1000" \
        -f target/<your_target>.cfg
```

Start at 1 MHz and raise `adapter speed` after the scan chain is found. The probe has been verified with the Raspberry Pi target over JTAG, on the same cable as the Pico build; the target configuration requirements are the target's, not the probe's (for a Raspberry Pi, `enable_jtag_gpio=1` in `config.txt`, and a TRST pull-up if the probe's nTRST is not wired).

If several CMSIS-DAP probes are attached, select this one by serial. The serial is shown by `lsusb -v` (iSerialNumber) and is a config-mode setting, so it must be given before `init`:

```
openocd -f interface/cmsis-dap.cfg \
        -c "adapter serial <serial-number>" \
        -c "transport select jtag" \
        -f target/<your_target>.cfg
```

or, in an OpenOCD script:

```
adapter serial <serial-number>
```

`reset_config none` is a safe choice with this firmware, because `DAP_SWJ_Pins` cannot drive TRST/SRST (section 9).

6.4 UART bridge

The CDC port is bridged to UART1:

Signal	GPIO	Header position	Connect to target
UART TX	GP4	6	target UART RX
UART RX	GP5	7	target UART TX
GND	—	8 (or 3)	target GND

Default format: 115200 8N1. The line coding requested by the host on the CDC port (baud rate, parity, data bits, stop bits) is applied to UART1 dynamically, so `stty` /your terminal program can change it. DTR/RTS/CTS are not wired by default (the board configuration does not define `PROBE_UART_DTR`, `PROBE_UART_RTS` or `PROBE_UART_CTS`).

Example:

```
picocom -b 115200 /dev/ttyACM0
```

7. Differences from `JTAGprobe_smp_pio_std`

7.1 Source tree

A recursive diff of the two trees (excluding `build/` and `.git/`) reports **only** `CMakeLists.txt` as different. All of `src/`, `include/`, the PIO program, the µOS++ port and the CMSIS-DAP code are identical. With the fix described in this document, `include/DAP_config.h` also differs from the Pico tree by the small `DAP_SETUP()` addition (section 7.4); it is harmless and appropriate for both boards.

7.2 CMake differences

Item	`JTAGprobe_smp_pio_std` (Pico)	`JTAGprobe_smp_pio_std_weact`	Effect
`PICO_BOARD`	`pico` (SDK default)	`weact_studio_rp2040_16mb`	16 MB flash, WeAct pin defaults
`PICO_STDIO_UART`	SDK default **ON**	now **ON** as well (the `OFF` override was removed)	

Item	`JTAGprobe_smp_pio_std` (Pico)	`JTAGprobe_smp_pio_std_weact`	Effect
			keeps UART0 on GP0/GP1, nTRST high from boot
Target name	`JTAGprobe_smp_pio_std`	`JTAGprobe_smp_pio_std_weact`	output file names only
Everything else	—	identical	PIO, libs, defines, `copy_to_ram`

The `PICO_STDIO_UART OFF` override existed only in this tree and was the reason the WeAct build failed; see section 8.

7.3 Compile-line differences

Comparing the Ninja compile definitions of `src/pio_jtag.c` in the two build directories, the only differences are:

```
< PICO_BOARD="pico"                          > PICO_BOARD="weact_studio_rp2040_16mb"
< LIB_PICO_STDIO_UART=1                      (removed in the old WeAct config)
< PICO_TARGET_NAME="JTAGprobe_smp_pio_std"   > PICO_TARGET_NAME="JTAGprobe_smp_pio_std_weact"
```

The generated `jtag.pio.h` is byte-identical. The first 252 bytes of both `.bin` images (the boot2 second stage) are identical, which confirms the same `W25Q080` boot2 and the same `PICO_FLASH_SPI_CLKDIV`.

7.4 `include/DAP_config.h`

Only `DAP_SETUP()` differs (7 added lines, shown in section 4.4):

- `gpio_pull_up(PROBE_PIN_nRESET)` — keep nRESET released from boot.
- Drive `PROBE_PIN_nTRST` high (latch, then output enable).

No pin number, mask or helper changed. `DAP_SWJ_*` in `include/DAP.h` is also unchanged (still the GPIO numbers, matching the working Pico tree).

7.5 Hardware differences

Feature	Raspberry Pi Pico	WeAct RP2040 16 MB	Impact on the probe
Flash	2 MB (SDK default; 4 MB on some boards)	16 MB	`PICO_FLASH_SIZE_BYTES`; irrelevant at runtime (`copy_to_ram`)
Boot2 / QSPI divider	W25Q080, clkdiv 2	same	identical boot2
USB connector	micro-USB	USB-C	none
3.3 V supply	SMPS	SC662K LDO, 250 mA	watch the target's current if powered from 3.3 V
`3V3_EN`	yes	no (NC)	none for the probe
Right header	Pico mapping	3V3/GP24/GP29/GP26 shifted	do not use a Pico adapter for 3.3 V
Left header pins 1–10	GP0…GP7	identical	none; same ribbon works
LED	GP25, green	GP25, blue, 1 kΩ	same firmware
`GP23`	SMPS PS	`KEY` button	firmware does not drive it

Feature	Raspberry Pi Pico	WeAct RP2040 16 MB	Impact on the probe
`GP24`	VBUS sense	on the header	unrelated
Debug header	3 pins	4 pins (3V3, SWDIO, SWCLK, GND)	do not plug a Pico SWD cable
Size	21 × 51 mm	21.52 × 53 mm	2 mm overhang
Crystal	12 MHz	12 MHz	same clocks

The probe's signals (GP0–GP7 and GP25) are electrically identical on both boards. The only firmware-visible hardware difference is the flash size.

7.6 Toolchain and binary differences

Item	Pico build	WeAct build
Compiler (`.comment`)	Arch GCC 16.2.0 / 16.1.0	xPack GCC 15.2.1 (20251203)
SDK	2.3.0	2.3.0
`PICO_BOARD`	`pico`	`weact_studio_rp2040_16mb`
UF2 / BIN	122,368 / 61,168 bytes	120,320 / 60,108 bytes
`pio_jtag_init` size	0x300	0x264
`pio_jtag_sequences` size	0x2d0	0x2b0

The different function sizes are ordinary inlining/scheduling differences between GCC versions; the two builds are semantically equivalent in the JTAG path. They are **not** a controlled comparison: the Pico binary was produced with a different compiler (and that compiler is no longer installed on this machine). For a clean A/B, build both trees with the same toolchain. During the investigation the same WeAct configuration was also built with xPack GCC 14.2.1; the final firmware uses 15.2.1 as requested.

7.7 What is identical

- Pin numbers and roles: nTRST GP0, nRESET GP1, TCK GP2, TMS GP3, UART1 GP4/GP5, TDI GP6, TDO GP7, LED GP25.
- The PIO program and the `DAP_JTAG_Sequence` streaming path.
- The CMSIS-DAP command layer, SWD path, SWO code paths, USB descriptors and VID:PID `2e8a:000c`.
- The µOS++ SMP split: USB/UART on core 0, DAP on core 1.
- The UART bridge behaviour and defaults.
- The status-LED behaviour.
- The `copy_to_ram` binary type.

8. Root cause of the original failure and the fix

8.1 Symptom

With the first WeAct build, OpenOCD connected to the probe and read the pin levels, then failed to see any TAP:

```
Info : CMSIS-DAP: Interface Initialised (JTAG)
Info : SWCLK/TCK = 1 SWDIO/TMS = 1 TDI = 1 TDO = 1 nTRST = 1 nRESET = 1
Info : CMSIS-DAP: Interface ready
Info : clock speed 4000 kHz
Info : cmsis-dap JTAG TLR_RESET
Info : cmsis-dap JTAG TLR_RESET
Error: JTAG scan chain interrogation failed: all ones
Error: bcm2837.cpu: IR capture error; saw 0x0f not 0x01
```

The probe's outputs were correct at the pads, and the same cable, target and OpenOCD configuration worked with the Pico build. "All ones" means the target never drove TDO: its TAP was held in Test-Logic-Reset, so TDO stayed tri-stated and the probe's pull-up read 1 on every bit.

8.2 Mechanism

The CMSIS-DAP layer in this firmware has no host-controlled nTRST: `PIN_nTRST_OUT()` is an empty stub and `PIN_nTRST_IN()` returns 0 (`include/DAP_config.h`; see section 9). nTRST is therefore whatever the firmware's own GPIO code leaves on GP0.

The working Pico build never exposes this because of an accident:

1. `PICO_STDIO_UART` defaults to ON, so TinyUSB's `board_init()` calls `stdio_uart_init_full(uart0, 115200, 0, 1)`.
2. Both board headers put UART0 TX on **GP0** and RX on **GP1**.
3. UART0 TX idles **high**, so GP0 — nTRST — is driven high from `board_init()` until `pio_jtag_init()` later re-muxes it to SIO (also high). The target's TAP is released for the whole session.

The first WeAct `CMakeLists.txt` contained:

```
set(PICO_STDIO_UART OFF CACHE BOOL "..." FORCE)
```

That was added to "stop `board_init()` from hijacking GP0/GP1". It removed the UART0 setup, so on the WeAct:

- after `DAP_SETUP()`, GP0 was an input with the RP2040 pad reset pull-down (the pad default), i.e. **nTRST asserted**;
- it stayed asserted from power-up until OpenOCD sent `DAP_Connect` and `pio_jtag_init()` drove it high;
- the target saw nTRST low for the whole boot/idle period, and its TAP was in Test-Logic-Reset when the scan started.

The JTAG sequencer itself (PIO, TMS/TDI stream, TDO capture) was never at fault. That is why bench tests of `DAP_JTAG_Sequence` showed the correct pin activity while OpenOCD still reported all ones: the probe's clocks and data were fine, the target was simply held in reset.

8.3 The fix

Two changes, both in this tree:

1. **`CMakeLists.txt`** — removed the `PICO_STDIO_UART OFF` override, so the build behaves exactly like the working Pico build: `board_init()` configures UART0 on GP0/GP1 and nTRST is high from boot.
2. **`include/DAP_config.h`** — `DAP_SETUP()` now drives nTRST (GP0) high explicitly and enables the pull-up on nRESET (GP1). This makes the idle pin state correct even if the stdio configuration changes again, and it releases nRESET from boot instead of leaving it pulled down.

No pin number, PIO program or protocol code was changed.

8.4 Why the bench tests looked fine

The previous investigation drove `DAP_JTAG_Sequence` directly and read back the probe's own pads: TCK/TMS/TDI toggled correctly, TDO read 1. That is consistent with a healthy probe driving a target whose TAP is in reset. The "GP0/GP1 floating after PORT_OFF" observation and "TDO always 1" are also consistent with a connected target in reset, so they did not prove a wiring fault. The decisive difference was the probe's idle nTRST state, which the Pico build provides through UART0 and the WeAct override removed.

8.5 Verification

After rebuilding with GCC 15.2.1:

- `board_init` contains a call to `stdio_uart_init_full` and `LIB_PICO_STDIO_UART=1` is present;
- `DAP_Setup` writes the GP0 latch and output enable high and calls `gpio_set_pulls(1, up=1, down=0)` for GP1;
- on hardware, JTAG works with OpenOCD on the same target and cable as the Pico build, and the UART bridge works.

Expected OpenOCD behaviour now: the scan chain is found, the target's IDCODE is reported, and JTAG operations proceed. The `SWCLK/TCK = ...` log line is still not in CMSIS-DAP spec order because of `DAP_SWJ_*` (section 9); that is cosmetic and does not affect the scan.

8.6 Rules to avoid a regression

- Do not disable the SDK stdio UART on this design.
- If a JTAG probe firmware ever needs to run with stdio UART off, it must drive nTRST high itself, as `DAP_SETUP()` now does.
- When comparing two builds, build them with the same toolchain and check the compile definitions, not just the source tree.

9. Known limitations

These are inherited from the Pico firmware and are documented here so that they are not mistaken for WeAct-specific problems.

1. **`DAP_SWJ_Pins` bit layout.** The CMSIS-DAP specification defines the select/value/response bits as TCK 0, TMS 1, TDI 2, TDO 3, nTRST 5, nRESET 7. `include/DAP.h` uses the GPIO numbers (TCK 2, TMS 3, TDI 6, TDO 7, nTRST 0, nRESET 1) as bit positions. Consequences: OpenOCD's nTRST/nRESET writes through `DAP_SWJ_Pins` do nothing, and the returned pin-status byte is not in spec order. The bit positions were changed to spec order once (commit `01685db`) and reverted (`5c31a12`) because SWD then read DPIDR `0x00000000`; do not "fix" this without retesting SWD.
2. **nTRST is not host-controllable.** GP0 is driven high by `DAP_SETUP()` and `pio_jtag_init()`; no DAP command can pulse it. Use a TMS-based TAP reset (five TCK with TMS high), which OpenOCD does by default.
3. **nRESET is effectively not host-controllable either**, for the same bit mapping reason. `reset_config none` or a target-side software reset is the reliable option.
4. **Bit-bang timing assumes a 125 MHz system clock.** `DAP_JTAG_Sequence` uses the actual `clock_get_hz(clk_sys)` for the PIO divider, but the bit-banged paths use a 125 MHz constant. If the board is ever run at another frequency, the PIO JTAG rate stays correct while SWD and the non-sequence JTAG commands scale.
5. **Mixed clock rates within one JTAG session.** `DAP_JTAG_Sequence` runs at the PIO rate; `DAP_Transfer` / `DAP_TransferBlock` bit-bang at the slower rate. Both are valid JTAG. Only throughput differs.
6. **Product string.** It still says "JTAG probe on Pico (CMSIS-DAP)". It is cosmetic; change `PROBE_PRODUCT_STRING` in `include/board_pico_config.h` if a distinct name is wanted.

7. **Leftover non-threaded path.** `os_main()` still contains a `THREADED = 0` loop that polls IDCODE and sends HID reports; it is dead code in this configuration.

10. Troubleshooting

Symptom	Check
OpenOCD prints “all ones” / IR capture error	Target TAP in reset or TDO not reaching the probe. Verify nTRST (GP0) is high after boot; on a Raspberry Pi target set `enable_jtag_gpio=1` and provide a TRST pull-up if the probe’s nTRST is not wired. Check the TDO wire on position 10.
OpenOCD reports a different scan chain than expected	Confirm the 10-way cable is on `J1` positions 1–10, in the same orientation as on the Pico. The left header is position-compatible; the right header is not.
Probe not found (`unable to find a matching CMSIS-DAP device`)	`lsusb` for `2e8a:000c`; check the USB-C cable (data, not charge-only); reflash; check libusb/udev permissions. If several probes are attached, select by serial.
CDC port appears but no UART data	TX/RX must cross (GP4 → target RX, GP5 ← target TX); common GND; matching baud (host line coding is applied to UART1); target UART enabled.
LED never blinks during JTAG activity	The DAP thread is not processing commands (USB/DAP path) or the wrong image is flashed. The LED is steady on when idle.
Build still behaves like the old one	Stale cache: `grep PICO_STDIO_UART build/CMakeCache.txt` must be `ON`; `LIB_PICO_STDIO_UART=1` must be in `build.ninja`. Otherwise reconfigure or delete `build/`.
JTAG works, SWD does not	SWD does not use the PIO and is subject to the `DAP_SWJ_Pins` caveat; use `transport select swd` and a low `adapter speed` while diagnosing.
Scan works at a low speed but not at a high one	Lower `adapter speed`; the target and the wiring set the limit, not the probe.
3.3 V adapter does not power the target	On the WeAct, 3.3 V is at right-header position 4, not position 5 (which is GPIO24).

Appendix A — exact comparison commands

These are the commands used to establish the differences in section 7.

```
cd /home/dan/Downloads/pico-jtag

# 1. Source trees
diff -rq JTAGprobe_smp_pio_std JTAGprobe_smp_pio_std_weact \
     --exclude=build --exclude=.git
# -> only CMakeLists.txt differs (and untracked docs/build dirs)

# 2. Compile definitions actually used
diff <(grep -o 'PICO_[A-Z0-9_]*=[^ ]*' \
        JTAGprobe_smp_pio_std/build/build.ninja | sort -u) \
     <(grep -o 'PICO_[A-Z0-9_]*=[^ ]*' \
        JTAGprobe_smp_pio_std_weact/build/build.ninja | sort -u)
# -> PICO_BOARD, PICO_STDIO_UART/LIB_PICO_STDIO_UART, PICO_TARGET_NAME

# 3. Compilers
readelf -p .comment JTAGprobe_smp_pio_std/build/JTAGprobe_smp_pio_std.elf
readelf -p .comment JTAGprobe_smp_pio_std_weact/build/JTAGprobe_smp_pio_std_weact.elf

# 4. Boot2 (first 256-byte stage)
cmp -n 252 JTAGprobe_smp_pio_std/build/JTAGprobe_smp_pio_std.bin \
         JTAGprobe_smp_pio_std_weact/build/JTAGprobe_smp_pio_std_weact.bin
# -> identical

# 5. Generated PIO header
diff JTAGprobe_smp_pio_std/build/jtag.pio.h \
     JTAGprobe_smp_pio_std_weact/build/jtag.pio.h
# -> identical
```

Appendix B — build-definition reference

Grouped view of the definitions that matter (all confirmed in `build/build.ninja`):

```
Board / platform
  PICO_BOARD="weact_studio_rp2040_16mb"
  PICO_RP2040=1
  PICO_COPY_TO_RAM=1
  PICO_NO_FLASH=0
  PICO_FLASH_SIZE_BYTES=16777216
  PICO_BOOT_STAGE2_CHOOSE_W25Q080=1
  PICO_FLASH_SPI_CLKDIV=2

USB
  CFG_TUSB_MCU=OPT_MCU_RP2040
  CFG_TUSB_OS=OPT_OS_NONE
  PICO_RP2040_USB_DEVICE_ENUMERATION_FIX=1
  PICO_RP2040_USB_DEVICE_UFRAME_FIX=1

RTOS
  OS_USE_OS_APP_CONFIG_H
  OS_USE_SMP_SCHEDULER=1
  OS_NCPU=2
  PICO_USE_MALLOC_MUTEX=0

Stdio / target
  LIB_PICO_STDIO_UART=1
  PICO_TARGET_NAME="JTAGprobe_smp_pio_std_weact"
```

The PIO JTAG program is compiled from `src/jtag.pio` by the SDK's pioasm; the generated header is identical to the Pico build's.

Appendix C — files

```
JTAGprobe_smp_pio_std_weact/
├── CMakeLists.txt                  board, target, build type
├── include/
│   ├── board_pico_config.h         LED 25, UART1 GP4/GP5, product string
│   ├── DAP_config.h                pins, PIO/SIO helpers, DAP_SETUP (nTRST/nRESET)
│   ├── DAP.h                       CMSIS-DAP definitions, DAP_SWJ_* (GPIO numbers)
│   └── pio_jtag.h                  PIO JTAG state (pins, clock)
├── src/
│   ├── jtag.pio                    PIO program (TCK/TMS/TDI/TDO)
│   ├── pio_jtag.c                  PIO setup and sequence streaming
│   ├── DAP.c, SW_DP.c, JTAG_DP.c   CMSIS-DAP command layer
│   ├── main.c                      threads, os_main, DAP_Setup/board_init order
│   ├── cdc_uart.c                  UART bridge (UART1, line coding)
│   ├── led.c / led.h               GP25 status LED
│   ├── usb_descriptors.c           VID:PID 2e8a:000c, product string
│   ├── get_serial.c                flash unique ID -> USB serial
│   ├── tusb_edpt_handler.c         CMSIS-DAP v2 bulk transport, 512-byte packets
│   └── tusb_config.h               TinyUSB configuration
├── micro-os-plus/                  µOS++ III SMP kernel and RP2040 port
├── CMSIS_6/                        CMSIS core headers
├── weact.md / weact.pdf            earlier investigation notes (superseded)
├── weact_style.css                 stylesheet for this document
└── WEACT_CONFIGURATION_BUILD_AND_DIFFERENCES.md/.pdf   this document
```

Appendix D — references

- WeAct Studio RP2040 Core Board V1.0: `/home/dan/Downloads/PICO/WeActStudio.RP2040CoreBoard-main/`, especially `HDK/PINout.PNG`, `HDK/T.png`, `HDK/B.png` and `HDK/sch/WeAct-RP2040-V10 SchDoc.pdf`.
- Raspberry Pi Pico SDK 2.3.0: `/home/dan/PICO/pico-sdk` (commit `98a542c1`).
- RP2040 datasheet (QSPI, PIO, GPIO pads, pad reset state) and Raspberry Pi Pico datasheet, both in `/home/dan/Downloads/PICO/WeActStudio.RP2040CoreBoard-main/Doc/`.
- CMSIS-DAP specification (DAP_SWJ_Pins bit positions, DAP_JTAG_Sequence).
- OpenOCD 0.12 user's guide (`interface/cmsis-dap.cfg`, `transport select jtag`, `adapter speed`, `reset_config`).
- Project documents in this directory: `README.md`, `ARCHITECTURE_AND_IMPLEMENTATION.md`, `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md`.